Deep Research: Avatrada 57 Topic 025

Engineering Report: Portfolio Net Greeks Risk Check System

Authored By: Autonomous Technical Researcher **Date:** October 26, 2023
Subject: Deep-Dive Analysis of Portfolio-Level Net Gamma and Vega Risk Limits

This report provides a detailed engineering analysis of the specified pre-trade risk check for Portfolio Net Greeks. The analysis covers the deconstruction of the underlying financial mathematics, a practical implementation strategy, and a critical review of potential failure modes and optimizations.

---

1. Technical Deconstruction

The system component is a pre-trade risk management gateway designed to enforce hard limits on the portfolio's net Gamma and Vega exposure. Its primary function is to prevent trades that would push the portfolio's risk profile beyond predefined thresholds relative to its Net Asset Value (NAV) or Net Liquidation Value (NLV).

1.1 Core Concepts

- **Portfolio Net Greeks:** The risk of a portfolio of options is not simply the sum of the individual option risks. It is the aggregate sensitivity to market changes.
    - **Net Gamma (Γ_net):** The sum of the Gamma of all positions in the portfolio. It measures the rate of change of the portfolio's total Delta. A positive net gamma portfolio gains delta as the underlying rises and loses delta as it falls, which is generally a favorable convexity profile.

◦ **Net Vega (ν_net):** The sum of the Vega of all positions. It measures the portfolio's sensitivity to a 1% change in implied volatility (IV). A positive net vega portfolio gains value when IV increases.

- **Risk Limit Specification:** The system enforces two distinct limits:

    1. **Gamma Limit:** The portfolio's Dollar Gamma exposure must not exceed 0.5% of its NAV.
    2. **Vega Limit:** The portfolio's Dollar Vega exposure must not exceed 1.0% of its NAV.

- **Blocking Mechanism:** The system is designed to block only "long-gamma" trades if the Gamma limit is already breached. This means it will prevent any new trade that *increases* the portfolio's net gamma (e.g., buying options, closing short option positions) but will still allow trades that reduce it (e.g., selling options, closing long option positions).

1.2 Mathematical Formulation

The prompt requires a precise method to convert standard option greeks into a portfolio-level dollar exposure figure.

1.2.1 Portfolio Dollar Gamma Exposure

The core formula provided is `Sum(Position_Gamma * 0.5 * Spot^2)`. Let's deconstruct this. This formula is derived from the second-order term of the Taylor series expansion for an option's price change:

```
ΔPrice ≈ (Delta * ΔS) + (0.5 * Gamma * (ΔS)^2) + ...
```

Where: * `ΔPrice` is the change in the option's price. * `ΔS` is the change in the underlying's spot price.

The formula `0.5 * Gamma * Spot^2` calculates the P&L impact from Gamma for a change in the underlying equal to its entire spot price (`ΔS = Spot`). This represents an extreme, 100% move in the underlying. While a valid stress test, a more common industry standard is to measure the P&L impact for a 1% move. However, adhering to the prompt's specification, the calculation is as follows:

Step-by-Step Calculation:

1. **Option Gamma (Γ_opt):** This is the standard output from a pricing model (e.g., Black-Scholes). It is typically expressed "per share" (i.e., the change in an option's delta for a $1 move in the underlying).

2. **Position Gamma (Γ_pos):** This scales the option gamma by the size of the position.

    - `Multiplier`: The number of shares one option contract controls (typically 100 for US equities).
    - `Num_Contracts`: The number of contracts in the position (positive for long, negative for short).

    `math Γ_pos = Γ_opt * Num_Contracts * Multiplier`

3. **Position Dollar Gamma (Γ_$):** This converts the position gamma into the specified dollar exposure figure.

    - `S`: The current spot price of the underlying asset.

    `math Γ_$ = Γ_pos * 0.5 * S^2`

4. **Portfolio Net Dollar Gamma (Γ_$,net):** This is the sum of the Dollar Gamma for all `i` open option positions in the portfolio.

    `math Portfolio_Net_Dollar_Gamma = Σ (Γ_$,i) for all i in Portfolio`

1.2.2 Portfolio Dollar Vega Exposure

The Vega calculation is more direct. Standard Vega represents the dollar change in an option's price per 1% (or 1 vol point) change in implied volatility.

1. **Option Vega (ν_opt):** The standard output from a pricing model, per share.

2. **Position Dollar Vega (ν_$):** The total dollar sensitivity of the position to a 1% volatility change.

`math ν_$ = ν_opt * Num_Contracts * Multiplier`

3. **Portfolio Net Dollar Vega (ν_$,net):** The sum of the Dollar Vega for all `i` open option positions.

   `math Portfolio_Net_Dollar_Vega = Σ (ν_$,i) for all i in Portfolio`

---

2. Implementation Strategy

This system should be implemented as a pre-trade risk gateway that intercepts orders before they are sent to an execution venue.

2.1 System Architecture

The risk check should be a service that sits between the Order Management System (OMS) and the execution connections.

```
+-----------------+     +-----------------------+     +-------------------+
| Order Source    |----->| Pre-Trade Risk Gateway  |----->| Execution Venue    |
| (e.g., OMS)     |      | (Gamma/Vega Check Logic)|      | (e.g., Exchange)   |
+-----------------+     +-----------------------+     +-------------------+
                         |
                         | Dependencies:
                         |--> Portfolio State Cache (Redis)
                         |--> Market Data Feed (Spot, IV)
                         |--> Pricing Engine (QuantLib)
                         |--> Account Service (NAV)
```

2.2 Data Requirements

The gateway requires real-time access to the following data: * **Portfolio State:** A complete list of all open positions, including instrument identifiers and quantities. An in-memory cache like Redis is ideal for low-latency access. * **Market Data:** Real-time spot prices and implied volatilities for all optionable underlyings in the portfolio. * **Account Data:** The current Net Liquidation Value

(NAV) of the portfolio. * **Proposed Order:** The details of the trade being checked (instrument, side, quantity).

2.3 Core Logic (Pseudocode)

The logic must perform a "pro-forma" or "what-if" analysis: what would the portfolio's risk be *if* this trade were executed?

```
# Required Libraries/Services:
# pricing_engine: A service to calculate greeks (e.g., using QuantLib)
# portfolio_cache: A service providing current positions
# market_data_feed: A service for real-time spot and IV
# account_service: A service for the latest NAV

# Constants
GAMMA_LIMIT_PCT = 0.005  # 0.5%
VEGA_LIMIT_PCT = 0.010   # 1.0%
CONTRACT_MULTIPLIER = 100

def calculate_dollar_gamma(gamma_per_share, quantity, spot_price):
    """Calculates Dollar Gamma based on the prompt's formula."""
    position_gamma = gamma_per_share * quantity * CONTRACT_MULTIPLIER
    return position_gamma * 0.5 * (spot_price ** 2)

def calculate_dollar_vega(vega_per_share, quantity):
    """Calculates Dollar Vega."""
    return vega_per_share * quantity * CONTRACT_MULTIPLIER

def check_portfolio_risk(proposed_trade):
    """
    Main pre-trade risk check function.
    Returns (is_approved, reason).
    """
    # 1. Get current portfolio state and market data
    current_positions = portfolio_cache.get_all_positions()
    nav = account_service.get_nav()

    # 2. Calculate current portfolio Net Dollar Gamma and Vega
    net_dollar_gamma = 0
```

```
    net_dollar_vega = 0
    for pos in current_positions:
        if pos.is_option():
            spot = market_data_feed.get_spot(pos.underlying)
            iv = market_data_feed.get_iv(pos.instrument_id)
            greeks = pricing_engine.get_greeks(pos.instrument_id, spot, iv)

            net_dollar_gamma += calculate_dollar_gamma(greeks.gamma,
pos.quantity, spot)
            net_dollar_vega += calculate_dollar_vega(greeks.vega, pos.quantity)

    # 3. Calculate the risk contribution of the proposed trade
    trade_spot = market_data_feed.get_spot(proposed_trade.underlying)
    trade_iv = market_data_feed.get_iv(proposed_trade.instrument_id)
    trade_greeks = pricing_engine.get_greeks(proposed_trade.instrument_id,
trade_spot, trade_iv)

    # Note: trade_quantity is positive for buy, negative for sell
    trade_dollar_gamma = calculate_dollar_gamma(trade_greeks.gamma,
proposed_trade.quantity, trade_spot)
    trade_dollar_vega = calculate_dollar_vega(trade_greeks.vega,
proposed_trade.quantity)

    # 4. Calculate pro-forma (hypothetical) portfolio risk
    pro_forma_gamma = net_dollar_gamma + trade_dollar_gamma
    pro_forma_vega = net_dollar_vega + trade_dollar_vega

    # 5. Apply the risk limit logic
    gamma_limit_value = GAMMA_LIMIT_PCT * nav
    vega_limit_value = VEGA_LIMIT_PCT * nav

    # Check Vega Limit (absolute value)
    if abs(pro_forma_vega) > vega_limit_value:
        return (False, f"REJECT: Pro-forma Vega {pro_forma_vega:,.0f} exceeds
limit {vega_limit_value:,.0f}")

    # Check Gamma Limit (as specified)
    # The rule is to block NEW LONG-GAMMA trades if the limit is breached.
    is_long_gamma_trade = trade_dollar_gamma > 0
```

```
    if is_long_gamma_trade and pro_forma_gamma > gamma_limit_value:
        
# We also check if the current gamma is already over the limit, as per a strict
interpretation.
        # A more lenient rule might allow trades as long as they don't push the
portfolio over the limit.
        # The prompt implies blocking if the *resulting* sum exceeds the limit.
        return (False, f"REJECT: Long-gamma trade blocked. Pro-forma Gamma
{pro_forma_gamma:,.0f} exceeds limit {gamma_limit_value:,.0f}")

    # 6. If all checks pass
    return (True, "APPROVED")
```

2.4 Recommended Libraries & Tools

- **Pricing Engine:** QuantLib (via Python bindings) is the industry standard for financial instrument pricing. It provides robust implementations of Black-Scholes-Merton and other models.
- **Data Caching:** Redis is an excellent choice for an in-memory store for portfolio positions and market data due to its high performance.
- **Numerical Computation:** NumPy should be used for all numerical calculations for performance and correctness. Portfolio-wide calculations can be vectorized to avoid slow loops.
- **Messaging:** A low-latency messaging system like ZeroMQ or a more robust one like Kafka can be used for distributing market data and order flow.

---

3. Critical Analysis

A robust system must account for potential failures, edge cases, and performance bottlenecks.

3.1 Potential Failure Modes & Edge Cases

- **Stale Data:** The biggest risk is acting on stale data. If the `NAV`, `Spot`, or `IV` data is delayed, the risk calculation will be inaccurate. This could lead to incorrectly blocking valid trades or, more dangerously, approving trades that violate risk limits.

    - **Mitigation:** Implement data health checks and heartbeats. The system should reject orders if market data or NAV timestamps are older than a configured threshold (e.g., 500ms).

- **The Dollar Gamma Formula:** The specified formula (`0.5 * Γ * S^2`) measures the P&L impact of a 100% move in the underlying. This is an extreme tail-risk scenario. For day-to-day risk management, a 1% move (`0.5 * Γ * (0.01 * S)^2`) is a more standard and sensitive metric. The current formula may be too blunt, only triggering in very high gamma scenarios.

    - **Recommendation:** Confirm the business intent of this formula. It may be intended as a disaster check, but a 1% gamma dollar metric should be calculated and monitored alongside it.

- **Correlated Risk:** The system sums greeks naively, assuming all underlyings are independent. A portfolio with large long gamma positions in both SPY and QQQ has significantly more correlated market risk than one with positions in SPY and a non-correlated asset.

    - **Mitigation (Advanced):** For a more sophisticated check, implement scenario-based analysis. For example, calculate the portfolio's P&L under a "-3% market move" scenario, using a beta-weighted approach for each underlying.

- **Intraday Volatility Skew Changes:** The Vega check assumes a parallel shift in the volatility surface. In reality, a market shock can cause the skew to twist (e.g., downside puts become much more expensive relative to upside calls). The single Vega number does not capture this risk.

    - **Mitigation (Advanced):** Implement risk limits based on "Vega buckets" (e.g., limit exposure to 30-day, 90-day, 1-year IV) or by using skew-aware models.

- **"Greeks of the Greeks":** Gamma is not constant. Its sensitivity to the underlying price is measured by `Speed`, and its sensitivity to volatility is measured by `Vomma` or `Volga`. In a volatile market, the initial Gamma calculation can become inaccurate very quickly.

- **Mitigation:** The risk check should be fast enough to run on every order. For very large or volatile moves, the system could trigger a full portfolio re-pricing to ensure greeks are fresh.

3.2 System Optimizations

- **Incremental Updates:** Instead of recalculating the entire portfolio's risk on every trade, maintain a cached value for `Portfolio_Net_Dollar_Gamma` and `Portfolio_Net_Dollar_Vega`. When a trade is checked, simply calculate the delta change from the proposed trade and apply it to the cached total. The cache is only fully rebuilt when positions are reconciled or on a periodic basis.
- **Vectorization:** Use NumPy to perform calculations. Load the entire portfolio's greeks, quantities, and underlying prices into NumPy arrays. The summation can then be performed with highly optimized vector operations, which is orders of magnitude faster than a Python loop.
- **Asynchronous Pricing:** The most latent part of the check is often the call to the pricing engine. The greeks for the portfolio can be pre-calculated and streamed to the risk gateway whenever market data changes beyond a certain threshold. This ensures that when an order arrives, the portfolio's current risk is already known, and only the greeks for the single proposed trade need to be calculated on the fly.